LAPORAN PRAKTIKUM
INTERNET OF THINKS 2

Oleh:

Nama : Salsabila Aulia Zahra

Npm 25782111

Kelas : 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

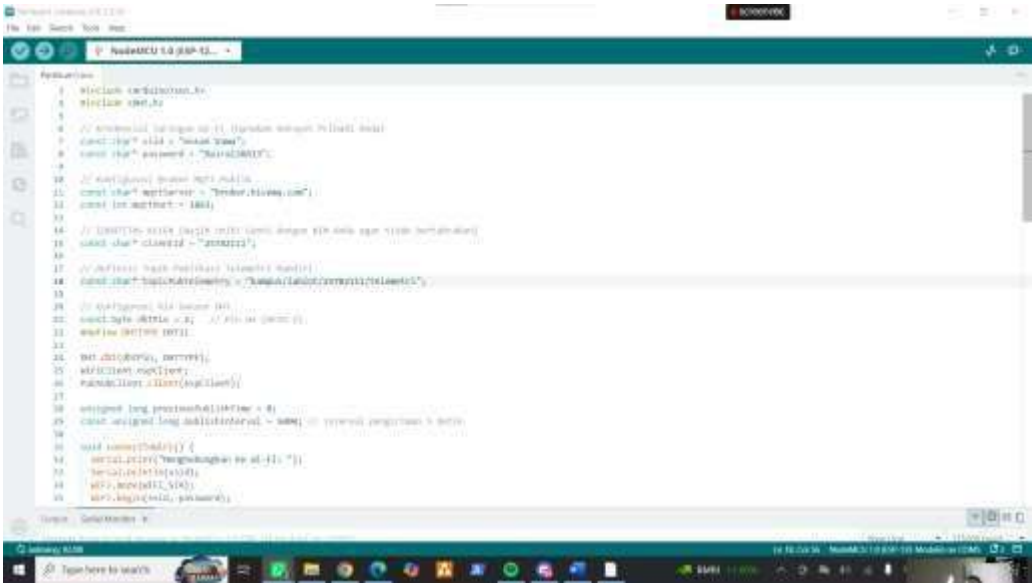
JURUSAN TEKNOLOGI INFORMASI

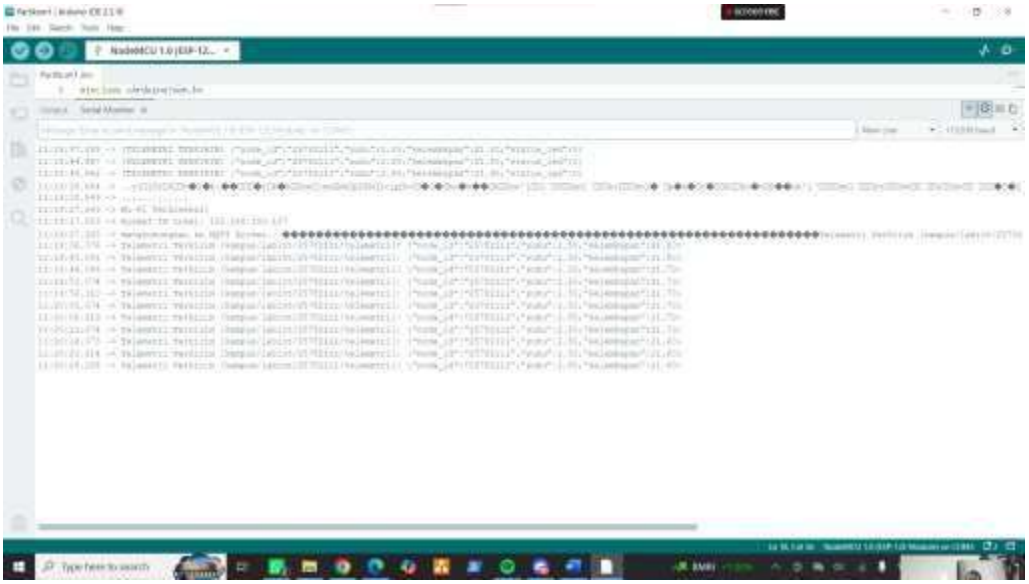
POLITEKNIK NEGERI LAMPUNG

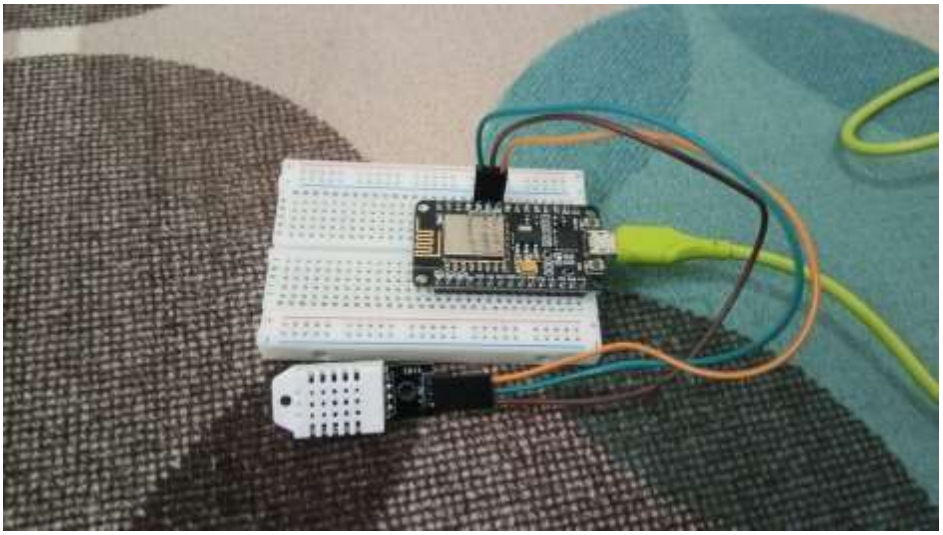
2026/2027

Praktikum 6

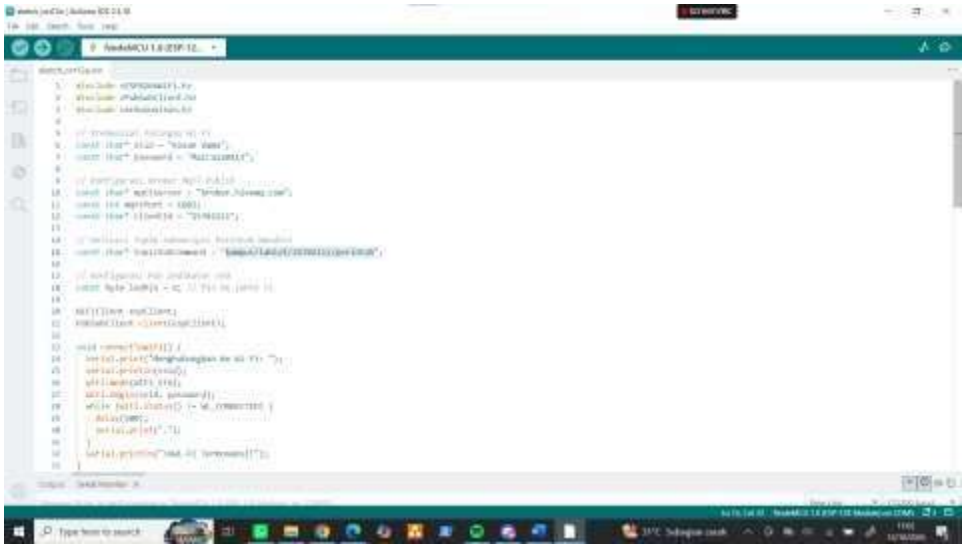
1. Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

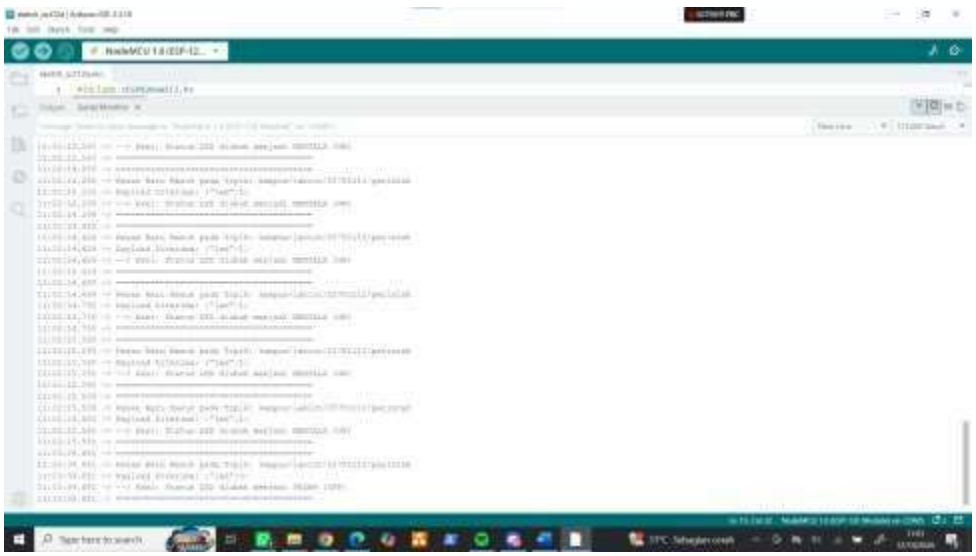


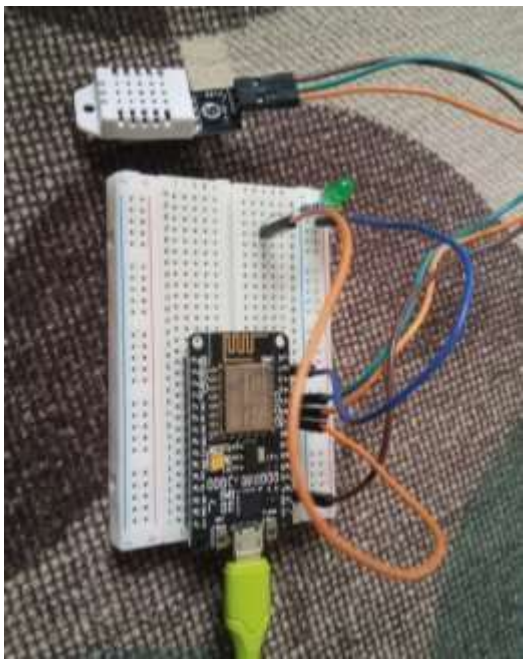




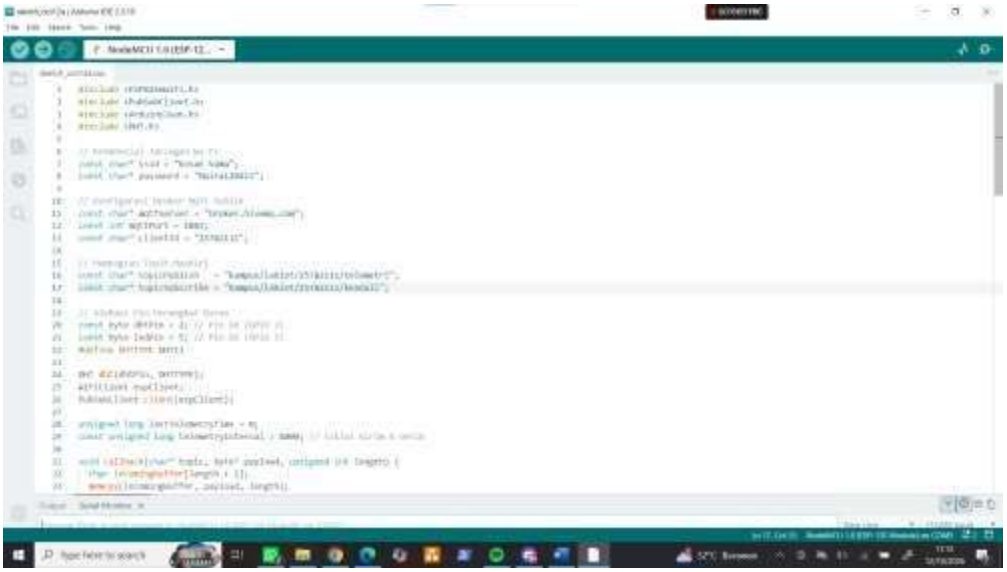
2. Subskripsi dan Kontrol Indikator LED Asinkron

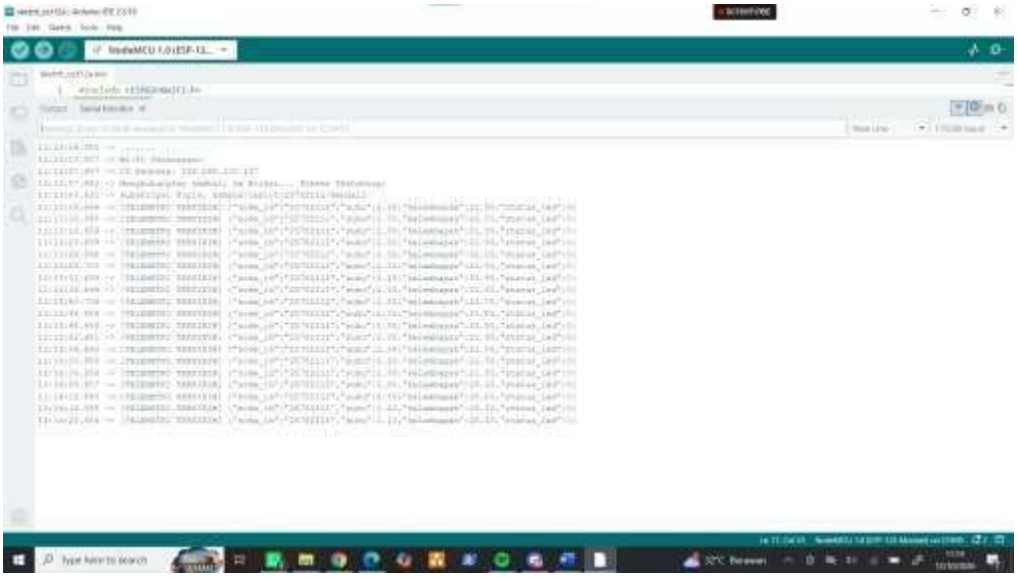


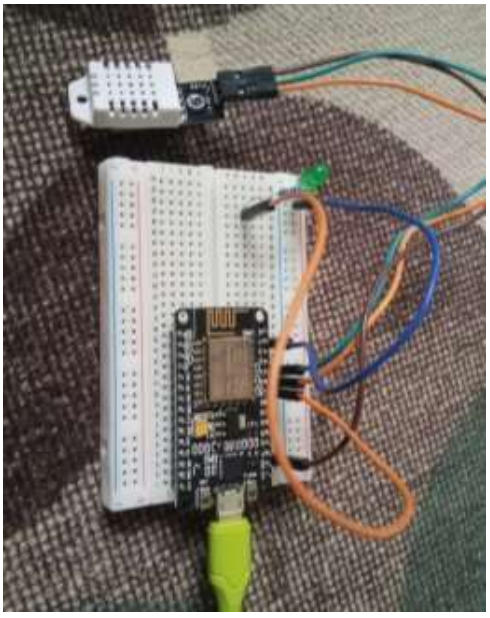




3. istem Terpadu Dua Arah (Full Duplex Telemetry & Control)







LATIHAN

- **Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):**
  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.

  MQTT lebih efisien dibandingkan HTTP karena ukuran pesan dan header MQTT relatif kecil. MQTT juga dirancang untuk mengirimkan data dalam jumlah kecil secara berkala, misalnya data suhu dan kelembapan dari sensor. Sementara itu, HTTP memiliki header permintaan dan respons yang cenderung lebih besar sehingga dapat menggunakan lebih banyak bandwidth.

  Pada jaringan seluler yang memiliki kuota terbatas atau koneksi tidak stabil, MQTT dapat mengurangi jumlah data tambahan yang dikirimkan. Dengan begitu, komunikasi perangkat IoT menjadi lebih hemat dan efisien.

  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.

  MQTT memiliki beban pemrosesan yang relatif ringan karena format pesannya sederhana dan komunikasi dilakukan melalui koneksi yang dapat dipertahankan tetap terbuka. Mikrokontroler tidak perlu membuat permintaan HTTP baru untuk setiap pertukaran data jika koneksi MQTT masih aktif.

  Selain itu, MQTT mendukung komunikasi dua arah melalui mekanisme publish-subscribe. Perangkat dapat mengirim data sensor sekaligus menerima perintah melalui koneksi yang sama. Hal ini membantu mengurangi pemrosesan komunikasi yang berulang, meskipun beban CPU sebenarnya tetap bergantung pada implementasi program dan kondisi jaringan.

  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

  MQTT lebih praktis untuk mengirim data ke banyak penerima karena menggunakan broker sebagai perantara. Perangkat pengirim cukup melakukan publish ke suatu topik, kemudian broker meneruskan pesan tersebut kepada semua perangkat yang melakukan subscribe pada topik yang sesuai.

  Contohnya, satu sensor suhu mengirimkan data ke topik pabrik/line1/m1/suhu. Data tersebut dapat diterima secara bersamaan oleh beberapa aplikasi pemantauan yang berlangganan

pada topik tersebut. Pada HTTP, pengiriman data ke banyak penerima biasanya membutuhkan permintaan atau mekanisme tambahan untuk menangani setiap penerima. Oleh karena itu, MQTT lebih sesuai untuk komunikasi IoT yang melibatkan banyak perangkat.

- **Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):**
  Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik `suhu`, `getaran`, dan `status`. Struktur topiknya adalah `pabrik/<line>/<mesin>/<sensor>`. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (`+` atau `#`) untuk:
  i. Memantau parameter `suhu` dari seluruh mesin di semua lini produksi.

     ```
     pabrik/+/+/suhu
     ```

     Penjelasan: wildcard `+` pertama mewakili seluruh lini produksi, seperti `line1`, `line2`, dan `line3`. Wildcard `+` kedua mewakili seluruh mesin, mulai dari `m1` sampai `m4`. Sementara itu, `suhu` membatasi data yang diterima hanya pada parameter suhu.

  ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin `m1` di lini `line2`.

     ```
     pabrik/line2/m1/+
     ```

     Penjelasan: topik ini membatasi pemantauan pada mesin `m1` di lini `line2`. Wildcard `+` di bagian terakhir memungkinkan penerimaan semua parameter sensor, yaitu `suhu`, `getaran`, dan `status`.

  iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

     ```
     pabrik/#
     ```

     Penjelasan: karakter `#` mencakup seluruh topik pada hierarki di bawah `pabrik/`. Dengan demikian, pelanggan yang berlangganan topik ini dapat menerima semua data dari seluruh lini, mesin, dan parameter sensor yang menggunakan awalan tersebut.

- **Latihan 3 (Implementasi Serialisasi Data Dinamis):**
  Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

  i. `"uptime_detik"`: Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus `millis() / 1000`).
  ii. `"ip_lokal"`: Alamat IP lokal mikrokontroler dalam format string (`WiFi.localIP().toString()`).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Untuk menambahkan informasi lama waktu aktif sistem dan alamat IP lokal, kita perlu menambahkan dua baris pada bagian penyusunan objek JSON di Praktikum 1.

Misalnya, kode sebelumnya sudah memiliki objek `doc` untuk menampung data telemetri. Tambahkan kode berikut:

```
doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();
```

Penjelasan:

1. `millis()` digunakan untuk mengetahui waktu yang telah berlalu sejak mikrokontroler mulai berjalan dalam satuan milidetik. Pembagian dengan `1000` mengubahnya menjadi detik.
2. `WiFi.localIP().toString()` mengambil alamat IP lokal ESP8266 yang sedang terhubung ke jaringan Wi-Fi, kemudian mengubahnya menjadi format string agar dapat dimasukkan ke dalam JSON.

Contoh hasil JSON yang dikirimkan:

```
{
  "suhu": 28.5,
  "kelembapan": 70.2,
  "uptime_detik": 125,
  "ip_lokal": "192.168.1.10"
}
```

- **Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):**
  Pada Praktikum 3, di dalam fungsi `loop()` terdapat pemanggilan rutin `client.loop()`.
  i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi `delay(6000)` di dalam fungsi `loop()`?

  Jika pengembang menambahkan delay(6000), ESP8266 akan berhenti menjalankan instruksi program berikutnya selama kurang lebih 6 detik. Akibatnya, pemanggilan client.loop() juga tertunda apabila berada setelah instruksi delay() tersebut.

  Padahal, client.loop() berfungsi untuk memproses komunikasi MQTT, termasuk memeriksa pesan masuk dan menjalankan fungsi callback() ketika ada perintah yang diterima. Karena itu, respons perangkat terhadap perintah dari broker akan menjadi lebih lambat. Misalnya, ketika pengguna mengirim perintah untuk menyalakan LED, LED tidak langsung merespons karena perangkat masih berada dalam kondisi menunggu.

  Selain itu, penundaan pemrosesan komunikasi dapat mengganggu pemeliharaan koneksi MQTT, terutama jika penundaan terlalu lama atau terjadi berulang kali. Jadi, penggunaan delay() yang terlalu

panjang sebaiknya dihindari pada program IoT yang membutuhkan respons cepat.

ii. Jelaskan bagaimana fungsi `reconnect()` bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Fungsi reconnect() digunakan untuk mencoba menghubungkan kembali perangkat ke broker MQTT ketika koneksi MQTT terputus. Umumnya, fungsi ini memeriksa apakah klien masih terhubung menggunakan client.connected(). Jika belum terhubung, program akan mencoba melakukan koneksi kembali melalui client.connect().

Jika koneksi berhasil, perangkat dapat melakukan subscribe() kembali ke topik perintah agar pesan yang dikirim melalui topik tersebut bisa diterima. Setelah itu, pemanggilan client.loop() akan membantu memproses komunikasi MQTT seperti biasa.

Namun, jika yang terputus adalah koneksi Wi-Fi, perangkat perlu memulihkan koneksi Wi-Fi terlebih dahulu sebelum mencoba menghubungkan kembali MQTT. Jadi, proses pemulihan bergantung pada jenis koneksi yang terputus.

Urutannya dapat dijelaskan sebagai berikut:

1. Perangkat memeriksa status koneksi MQTT.

2. Jika koneksi MQTT terputus, perangkat mencoba terhubung kembali ke broker.

3. Jika koneksi berhasil, perangkat berlangganan kembali ke topik yang diperlukan.

4. Jika Wi-Fi juga terputus, perangkat harus menghubungkan kembali Wi-Fi terlebih dahulu.

5. Setelah koneksi pulih, perangkat dapat kembali mengirim data sensor dan menerima perintah.

Kesimpulan: MQTT cocok digunakan pada sistem IoT karena komunikasi datanya relatif ringan, mendukung pengiriman pesan ke banyak penerima, dan memungkinkan perangkat mengirim maupun menerima data melalui mekanisme publish-subscribe. Agar sistem tetap responsif, program juga perlu menangani koneksi ulang dengan baik dan menghindari penggunaan delay() yang terlalu panjang.